ENJAY
Empowering Enterprises with Ennovations

Real-Time Call Activity Management System

Background

You are joining a B2B SaaS company that develops a CRM platform used by thousands of Indian SMBs. For many organizations, the Android application is the primary tool used by field sales executives and relationship managers to manage customer interactions.

A large portion of customer communication happens through regular phone calls. However, many of these conversations are never recorded in the CRM, resulting in incomplete customer history, missed follow-ups, poor reporting, and reduced sales productivity.

To address this challenge, the company is building a **Real-Time Call Activity Management System** that automatically detects phone calls, identifies whether they are business-related, and enables users to log them into the CRM with minimal effort.

This assignment is based on a real production problem that we solve for our customers.

Objective

Design and develop a production-ready Android solution capable of:

- Detecting phone call events in real time using Android's supported telephony and call log APIs.
- Capturing call details after the call is completed.
- Associating the call with an existing CRM Lead or Contact whenever possible.
- Allowing users to log business calls into the CRM.

- Respecting user privacy by ensuring personal calls are never uploaded.
- Synchronizing call activities reliably under real-world conditions.

The focus of this assignment is **engineering quality**, **architecture**, **problem solving**, and **production readiness**, rather than simply implementing a list of features.

Functional Requirements

Your application should support the following capabilities.

1. Real-Time Call Detection

Automatically detect:

- Incoming calls
- Outgoing calls
- Missed calls
- Rejected calls

Capture at minimum:

- Phone number
- Call type
- Call start time
- Call end time
- Call duration
- Timestamp

---

2. CRM Contact Matching

When a call is completed:

- Search for the phone number in CRM.
- If found, associate the activity with the matching Lead or Contact.

- If not found, allow the user to create a new Lead directly from the call.

---

3. Business vs Personal Calls

Not every call belongs in the CRM.

Provide an intuitive mechanism for users to distinguish between:

- Professional calls
- Personal calls

Requirements:

- Personal calls must never be synchronized with CRM.
- Professional calls should be available for CRM logging.
- The experience should minimize unnecessary user interaction.
- Explain the reasoning behind your chosen approach.

---

4. Post-Call Activity Logging

For business calls, allow users to record:

- Call notes
- Call outcome
- Follow-up date
- Next action
- Additional remarks (optional)

---

5. Offline Support

The application should continue working when:

- Internet is unavailable
- Network is unstable
- Sync fails

- Device restarts

Once connectivity is restored, pending activities should synchronize automatically.

---

6. Reliable Synchronization

Your synchronization mechanism should consider:

- Retry strategies
- Duplicate prevention
- Conflict handling
- Partial failures
- Network interruptions
- Idempotent uploads
- Efficient battery and network usage

Real-World Constraints

Assume this application will be used by thousands of sales representatives across India.

Your solution should consider:

- Low-end Android devices
- Poor or intermittent network connectivity
- Long offline periods
- Dual SIM devices
- Large numbers of daily calls
- OEM battery optimizations (Xiaomi, Oppo, Vivo, Realme, Samsung, etc.)
- Android background execution restrictions
- Different Android versions
- Privacy and security of locally stored data

ENJAY
Empowering Enterprises with Ennovations

What We Expect

We are intentionally **not prescribing any specific architecture, framework, library, or technology stack**.

Choose the tools, libraries, architectural patterns, and design principles that you believe are most appropriate for solving this problem.

More importantly, explain **why** you made those choices.

We are interested in your engineering decisions more than the specific technologies you use.

Deliverables

Please submit:

- Complete source code
- Build instructions
- APK (optional but preferred)
- README explaining setup
- Architecture overview
- Design decisions and trade-offs
- Assumptions made
- Testing approach
- Known limitations
- Future improvements